
EstateOS — Channel Setup Requests

What we need registered with Google, Microsoft, Meta and Twilio — and what to send back.

For the team registering EstateOS with Google, Microsoft, Meta and Twilio.

These are **platform-level registrations**, done once by us as the EstateOS operator. Customer organizations never register anything: after this is in place, a brokerage connects its mailbox or WhatsApp number by clicking Connect and signing in.

Send everything back as secrets (password manager or .env entries), never in chat or email body text. Nothing here is needed for website forms — those already work.

0. Context: what already works

Website forms — live today, no third party involved.

In EstateOS: Settings → Integrations → *Create a form endpoint* → name it. EstateOS mints a URL and key, and shows a copy-paste HTML form plus a JSON snippet. The customer's existing contact form can simply POST to the URL.

Tested end to end: submissions become a contact, a lead and an inbox thread; a repeat enquiry from the same email joins the existing person rather than duplicating them; unknown key returns 404; a submission with neither email nor phone returns 400; 20 submissions per minute per key are allowed; replayed webhooks are deduplicated by provider id.

Everything below is about the channels that *do* need external accounts.

1. Google Cloud — Gmail / Google Workspace

Who: whoever administers our Google Cloud organization.

Lead time: 2–6 weeks — Gmail scopes are "restricted" and need a security review by Google. Start this one first.

Steps

1. Google Cloud Console → create (or pick) a project named **EstateOS**.
2. APIs & Services → Library → enable **Gmail API** and **Google Calendar API**.
3. OAuth consent screen:
 - User type: **External**
 - App name: EstateOS, support email, logo (120×120 PNG)
 - Authorised domain: our production domain
 - Links: home page, privacy policy, terms of service — **all three must be live public URLs** or the review fails
4. Scopes — request exactly these, no more:
 - `https://www.googleapis.com/auth/gmail.readonly`
 - `https://www.googleapis.com/auth/gmail.send`
 - `https://www.googleapis.com/auth/gmail.labels`

- `https://www.googleapis.com/auth/calendar.events`
 - `openid, email, profile`
5. Credentials → Create credentials → **OAuth client ID** → Web application.
 - Authorised redirect URIs:
 - `https://<production-domain>/api/oauth/google/callback`
 - `https://<staging-domain>/api/oauth/google/callback`
 - `http://localhost:8080/api/oauth/google/callback`
 6. Submit for verification. Google will ask for a demo video of the OAuth flow and a justification for each scope — tell them: *"EstateOS is a real-estate CRM. It reads the brokerage's own shared mailbox to thread client enquiries into their inbox, and sends replies from that same address on the user's behalf."*

Send back

```
GOOGLE_OAUTH_CLIENT_ID=
GOOGLE_OAUTH_CLIENT_SECRET=
```

Faster interim option (no review, works this week)

If we don't want to wait for verification, each mailbox can connect over **IMAP/SMTP with an app password**: the user turns on 2-step verification, generates a 16-character app password, and pastes it into EstateOS. Slightly clunkier for them, no approval needed, works with any mail host. Say the word and I'll build this path first, then swap to OAuth when Google approves. If we take it, no Google registration is needed at all for email.

2. Microsoft Entra ID — Microsoft 365 / Outlook

Who: whoever administers our Microsoft tenant.

Lead time: 1–3 days — no lengthy review, but publisher verification helps avoid scary consent screens.

Steps

1. Entra admin centre → App registrations → **New registration**
 - Name: EstateOS
 - Supported account types: **Accounts in any organizational directory (multitenant)**
 - Redirect URI (Web):
 - `https://<production-domain>/api/oauth/microsoft/callback`
 - `https://<staging-domain>/api/oauth/microsoft/callback`
 - `http://localhost:8080/api/oauth/microsoft/callback`
2. Certificates & secrets → **New client secret** → 24-month expiry. Record the **Value** (not the ID) — it is shown once.
3. API permissions → Microsoft Graph → **Delegated**:
 - `Mail.ReadWrite, Mail.Send, Calendars.ReadWrite, User.Read, offline_access`
4. Complete **Publisher verification** (links the app to our verified Microsoft Partner account).

Send back

```
MICROSOFT_OAUTH_CLIENT_ID=      # Application (client) ID
MICROSOFT_OAUTH_CLIENT_SECRET=  # the secret Value
MICROSOFT_TENANT_ID=common      # keep as "common" for multitenant
```

3. Meta — WhatsApp Business Platform

Who: whoever owns our Meta Business account, plus someone who can supply company documents.

Lead time: 1–3 weeks — Business verification requires legal documents.

Steps

1. Meta Business Suite → create a **Business portfolio** for our company if we don't have one.
2. Business settings → **Business verification**: submit certificate of incorporation, proof of address, and a phone number Meta can call. This is the slow part; start it immediately.
3. developers.facebook.com → Create App → type **Business** → name EstateOS.
4. Add products: **WhatsApp** and **Facebook Login for Business**.
5. Configure **Embedded Signup** (this is what lets a brokerage connect their own number with a few clicks):
 - Valid OAuth redirect URI: `https://<production-domain>/api/oauth/meta/callback`
 - Configuration: WhatsApp Business Account + phone number selection
6. Webhooks → subscribe to the **messages** field, callback URL:
 - `https://<production-domain>/api/webhooks/whatsapp`
 - Verify token: invent a long random string and send it to me — I'll set the same value on our side.
7. App Review → request `whatsapp_business_management` and `whatsapp_business_messaging` with Advanced Access.
8. Note: each customer's own WhatsApp Business number must be one **not currently registered to the WhatsApp app** on a phone.

Send back

```
META_APP_ID=
META_APP_SECRET=
META_WEBHOOK_VERIFY_TOKEN=      # the random string from step 6
META_CONFIG_ID=                 # Embedded Signup configuration id from step 5
```

Faster alternative

A **Business Solution Provider** (Twilio, 360dialog, Wati) gets us sending in days instead of weeks, at a small per-message fee, and skips Meta app review entirely. The customer's experience is nearly identical. Worth considering if WhatsApp is urgent — many brokerages in Pakistan and the UAE will ask for it first.

4. Twilio — SMS and voice

Who: anyone with a company card.

Lead time: same day, except number provisioning in regulated markets (UAE and Pakistan need business documents and can take 1–3 weeks).

Steps

- 1. Create a Twilio account, upgrade from trial.
- 2. Buy one number per market we're selling into (US, UK, UAE, Pakistan) — check each country's regulatory bundle requirements.
- 3. Console → Account → API keys & tokens.
- 4. Messaging → set the inbound webhook to `https://<production-domain>/api/webhooks/twilio` (I'll confirm once built).

Send back

```

TWILIO_ACCOUNT_SID=
TWILIO_AUTH_TOKEN=
TWILIO_MESSAGING_SERVICE_SID=      # if a messaging service is used
```

5. Also needed from the team

Not integration keys, but blockers for the same work:

```

APP_URL=                # canonical production URL, used in OAuth redirects and invitation link
DATABASE_URL=           # production PostgreSQL (VPS, Vercel or Supabase – any plain Postgres)
SESSION_SECRET=         # 32+ random characters; encrypts stored channel credentials
ANTHROPIC_API_KEY=      # switches on the AI layer (summaries, reply drafting, matching explanations)
```

Plus, for the Google review specifically: a **live privacy policy page** and a **live terms of service page** on our production domain. Google will reject the app without them.

6. What happens once each arrives

We receive	I build	A customer then does
Google keys	OAuth flow, mailbox sync, send-as	Clicks Connect Gmail, signs in, picks the mailbox
Microsoft keys	Same for Graph	Clicks Connect Outlook, signs in, picks the mailbox
Meta keys	Embedded signup, inbound webhook, template handling, 24-hour session window	Clicks Connect WhatsApp, picks their number
Twilio keys	Inbound webhook, send path, number-to-office mapping	Picks which office owns which number
ANTHROPIC_API_KEY	Conversation summaries, reply drafting, match explanations, copilot	Nothing — it appears

Order I'd suggest: **Microsoft first** (fastest, unblocks email end to end), **Meta business verification started in parallel** (slowest), **Google submitted immediately after**, Twilio whenever.

Until a key lands, that channel's card in EstateOS says "Not built yet" and explains why. Nothing is ever shown as connected when it isn't.